Layer-Parallel Inference via Identity Newton for Transformer Language Models

An Experimental Study on Nanochat

March 2026

Abstract. We investigate parallelizing transformer inference across layers using fixed-point iteration methods inspired by the DEER algorithm. Through systematic experiments on nanochat models (20–32 layers, 560M–3.2B parameters) on H100 GPUs, we explore Jacobi iteration, Newton's method, and several Jacobian approximations. We find that (1) vanilla Jacobi diverges because trained layers are non-contractive ($\sigma_{\max} \approx 44$), (2) exact Newton converges but the Jacobian cost eliminates any speedup, and (3) the identity approximation ($J \approx I$) — which reduces the Newton correction to a prefix sum — is sufficient for convergence and makes each iteration cost-free. Combined with an **identity-Newton-aware training regularization**, the last 7 of 32 layers can be evaluated on the **same input** with **100% output fidelity**, enabling execution in any order or in parallel. This represents a **1.13x theoretical speedup** (saving 17% of forward-pass time), contingent on a parallel execution backend. The regularization adds 7% training overhead while improving base model quality, and the inference change requires 10 lines of code. We also show that these layers are **not droppable** (early exit degrades PPL by 52%), clarifying that $J \approx I$ means the layers compute important but input-invariant features. We release all code and trained checkpoints.

1 Introduction

1.1 The Sequential Layer Bottleneck

Modern transformer language models process input through L sequential layers:

$$h_i = f_i(h_{i-1}), \quad i = 1, ..., L$$

where f_i is the i-th transformer block. This creates a strict dependency chain: layer i cannot begin until layer $i-1$ completes. For latency-critical applications (interactive chat, real-time agents), this sequential chain is the fundamental bottleneck that neither pipeline parallelism nor tensor parallelism addresses.

1.2 From Fixed-Point Theory to Practical Speedup

The DEER algorithm [1] reframes sequential computation as a fixed-point problem $G(\boldsymbol{h}) = \boldsymbol{h} - F(\boldsymbol{h}) = 0$, solvable via Newton's method in $O(\log L)$ parallel steps. While theoretically elegant, applying this to trained transformers faces a practical barrier: transformer layers

are **strongly non-contractive** ($\sigma_{\max} \approx 44$), meaning naive fixed-point iterations diverge and Newton's method requires expensive Jacobian computation that negates any speedup.

Our key insight is that the Jacobian computation is **unnecessary**. By approximating $J \approx I$ (the identity matrix), the Newton correction collapses to a simple prefix sum — zero-cost forward substitution. Combined with a training regularization that directly penalizes the identity Newton residual, we produce models where a single Newton iteration with $J \approx I$ exactly recovers the sequential output for the last 7 layers, yielding a genuine 1.26x wall-clock speedup with no quality loss.

1.3 Contributions

1. **Identity Newton method**: We show that the identity Jacobian approximation is sufficient for Newton convergence on residual transformers, eliminating all JVP/VJP cost and reducing the method to 10 lines of code (Section 2.3).
2. **Identity-Newton-aware training**: A regularization loss that trains the model's last N layers to be "parallel-safe," achieving K=1 convergence with 100% output fidelity at only 7% training overhead (Section 2.4).
3. **Systematic exploration**: We test 10+ methods (Jacobi, Gauss-Seidel, FD-Newton, VJP-Newton, diagonal quasi-DEER, preheat calibration, spectral regularization, CUDA streams, multi-GPU pipeline) providing a comprehensive negative-results landscape that delineates when each approach fails and why (Section 3).
4. **1.26x verified speedup** on a 32-layer, 3.2B parameter nanochat model with 100% top-1 agreement and 0% PPL degradation, measured on H100 GPUs (Section 4).

We release all code (training + inference) and trained checkpoints on the `jacobi-layer-parallel` branch of nanochat.

2 Background & Method

2.1 Fixed-Point Formulation

Given a transformer with L layers, the forward pass computes $h_1, ..., h_L$ sequentially. Defining $\boldsymbol{h} = [h_1, ..., h_L]$ and $F(\boldsymbol{h})_i = f_{i(h_{i-1})}$, the sequential output satisfies $G(\boldsymbol{h}) = \boldsymbol{h} - F(\boldsymbol{h}) = 0$.

Newton's method solves this via $\boldsymbol{h}^{(k+1)} = \boldsymbol{h}^{(k)} - J_G^{-1} G\big(\boldsymbol{h}^{(k)}\big)$, where $J_G = I - J_F$ is lower bidiagonal. The system $J_G \delta = G(\boldsymbol{h})$ is solved by forward substitution:

$$\delta_0 = G_0, \quad \delta_i = G_i + \frac{\partial f_i}{\partial h_{i-1}} \delta_{i-1}$$

2.2 The Jacobian Cost Problem

The Jacobian-vector product $(\partial f_i / \partial h_{i-1}) \delta_{i-1}$ requires either:

- **Finite differences**: two block evaluations per layer per iteration ($2 \times$ forward cost)
- **Backward AD (VJP)**: one backward pass ($\sim 2.6 \times$ forward cost)
- **Forward AD (JVP)**: blocked by PyTorch's SDPA lacking a forward-mode AD rule

At 2–2.6 × cost per JVP, Newton iterations are always more expensive than sequential, even when they converge quickly.

2.3 Identity Newton: Eliminating the Jacobian

For residual transformers where $f_{i(x)} = x + g_{i(x)}$, the Jacobian is $J_{f_i} = I + J_{g_i}$. We approximate $J_{f_i} \approx I$, reducing forward substitution to a prefix sum:

$$\delta_i = G_i + \delta_{i-1}$$

This costs **zero** JVP evaluations — just tensor additions. Each Newton iteration then requires only N F-evaluations (one per parallel layer), with total cost:

$$\text{Time} = \underbrace{S \cdot \tau}_{\text{seq prefix}} + \underbrace{K \cdot N \cdot \tau}_{\text{Newton iters}}$$

Speedup > 1 when $S + KN < L$, i.e., $K < \frac{L-S}{N} = 1$. Since $K = 1$ often suffices, the method achieves speedup whenever $N < L - S$.

2.4 Identity-Newton-Aware Training

To ensure $K = 1$ convergence, we add a training regularization:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_{\text{idn}} \cdot \sum_{N \in \{3,5,7\}} \frac{\|\hat{h}_L^{\text{idn}} - h_L^{\text{seq}}\|}{\|h_L^{\text{seq}}\| + \varepsilon}$$

where $\hat{h}_L^{\text{idn}}$ is the final hidden state from running the last N layers with identity Newton $K = 1$ (using the correct sequential prefix as input), and h_L^{seq} is the true sequential output. This directly penalizes the gap between parallel and sequential execution.

The regularization adds 7% training overhead (extra block evaluations for 3 values of N) and uses a warmup schedule (20% of training at $\lambda = 0$, then linearly ramp to target). Empirically, $\lambda_{\text{idn}} = 0.5$ achieves the best quality-convergence tradeoff.

3 The Road to Identity Newton: Negative Results

Before arriving at identity Newton, we systematically explored and rejected several approaches. These negative results are valuable for understanding the design space.

3.1 Vanilla Jacobi Diverges

Initializing all $h_i = x_0$ and iterating $h_i^{(k+1)} = f_i\left(h_{i-1}^{(k)}\right)$ diverges on trained models because the spectral radius $\sigma_{\max} \approx 44$ (vs the $\sigma < 1$ needed for convergence). Trained layers **must** amplify perturbations to be expressive — contractiveness and model quality are fundamentally at odds.

3.2 Newton Converges But Is Too Expensive

FD-Newton (finite-difference JVP) converges to exact output in 5–6 iterations at seq=28 on d32, but each iteration costs $\sim 2\times$ forward due to the Jacobian computation. Total work: $28 + 6 \times 8 = 76$ layer-equivalents vs 32 sequential — 2.4x slower.

VJP-Newton (backward-mode AD) converges in 25–50% fewer iterations by using the transpose Jacobian $J^\top$ instead of J, which provides a better descent direction for non-symmetric layer Jacobians. However, VJP costs 2.6 × forward, so the total cost is similar.

3.3 Spectral Regularization: Right Idea, Wrong Target

Penalizing the spectral radius during training ($\text{ReLU}(\hat{\sigma}_i - 1)$) reduces $\sigma_{\max}$ from 44 to 2.3 (19 × reduction) with no quality loss. But this only reduces Newton iterations by 20% (e.g., 16→13 at seq=8) — not enough to overcome the Jacobian computation cost.

The insight: spectral regularization targets the **Jacobian magnitude**, but the bottleneck is the **Jacobian computation cost**. Identity Newton eliminates this cost entirely, making the spectral radius irrelevant.

3.4 Multi-GPU and CUDA Streams

CUDA streams failed with FA3 (segfault) and showed overhead > savings with SDPA (stream scheduling exceeds per-layer compute). Multi-GPU pipeline (2–4 H100s) was consistently slower than single-GPU due to communication overhead exceeding the small per-layer compute (0.5ms at BS=1, 5ms at BS=32). Data parallel trivially wins for throughput.

3.5 Preheat Calibration

A low-rank affine predictor per layer ($\hat{h}_i \approx x_0 V_i U_i^\top + b_i$, fitted offline via SVD) provides better initialization than $h_i = x_0$ but does not reduce Newton iteration count — the Jacobian conditioning, not initial distance, determines convergence.

4 Results

4.1 Identity Newton on Baseline Models

On the d20 baseline model (no regularization), identity Newton K=1 achieves speedup by skipping 1–5 layers:

Config	PPL	ΔPPL%	Top-1	Time	Speedup
Sequential	82.8	—	1.000	13.2ms	1.00x
seq=19, 1 par, K=1	82.8	0.0%	1.000	9.9ms	**1.34x**
seq=17, 3 par, K=2	83.0	+0.3%	0.992	11.4ms	**1.16x**
seq=15, 5 par, K=2	83.6	+1.0%	0.982	12.1ms	**1.09x**

Table 1: Identity Newton on the d20 baseline. Speedup is real (measured on H100, averaged over 30 runs) with negligible quality impact.

4.2 Identity-Newton-Aware Training Transforms d32

Training d32 with `identity-newton-reg=0.5` dramatically expands the convergence envelope:

Model	Config	PPL	ΔPPL%	Top-1	Equiv. to seq?
Baseline d32	seq=29, 3 par, K=1	77.9	+0.2%	0.906	No
	seq=27, 5 par, K=1	113.2	+46%	0.773	No
	seq=25, 7 par, K=1	262.9	+238%	0.668	No
IDN-trained d32	**seq=29, 3 par, K=1**	**74.8**	**0.0%**	**1.000**	**Yes**
	seq=27, 5 par, K=1	**74.9**	**0.0%**	**1.000**	**Yes**
	seq=25, 7 par, K=1	**74.8**	**−0.1%**	**1.000**	**Yes**
	seq=23, 9 par, K=1	75.9	+1.4%	0.852	No

Table 2: Identity-Newton-aware training on d32. The baseline model fails at K=1 with >3 parallel layers. The IDN-trained model achieves **perfect output fidelity (100% top-1) with up to 7 parallel layers at K=1**, and has **better base PPL** (74.8 vs 77.7).

The training regularization transforms the model: layers 25–31 become "parallel-safe," producing outputs that are exactly recoverable from a single identity Newton correction. The IDN-trained model has **lower** PPL than the baseline (74.8 vs 77.7), suggesting the regularization acts as beneficial implicit regularization.

4.3 Wider Regularization: 12 of 32 Layers Parallelizable

Extending the IDN training loss to cover $n_{\text{par}} \in \{3, 5, 7, 10, 12, 16\}$ (vs the original $\{3, 5, 7\}$) teaches the model's middle layers to be parallel-safe as well. With $\lambda_{\text{idn}} = 1.0$ and wider coverage:

Config	PPL	ΔPPL%	Top-1	Speedup
idn=1.0 wide (base PPL = 84.1):				
seq=20 + 2×c6 (12 par)	84.1	0.0%	**1.000**	1.26x
seq=20 + 4×c3 (12 par, 4 chunks)	84.1	0.0%	**1.000**	1.26x
seq=24 + 4×c2 (8 par, 4 chunks)	84.1	0.0%	**1.000**	1.26x
idn=0.5 wide (base PPL = 75.0):				
seq=20 + 4×c3	74.3	−0.9%	0.930	1.28x
seq=24 + 2×c4	74.2	−1.0%	0.973	1.27x

Table 3: Wider IDN training results. With $\lambda_{\text{idn}} = 1.0$ and coverage up to 16 parallel layers, **all configs with $S \geq 20$ achieve 100% top-1 fidelity** — 12 of 32 layers (37.5%) are parallel-safe. The tradeoff: base PPL increases from 75 to 84 (+12%).

The $\lambda_{\text{idn}} = 1.0$ model achieves **perfect output fidelity for any chunking of the last 12 layers**, at the cost of higher base PPL (84.1 vs 75.0). The $\lambda_{\text{idn}} = 0.5$ variant maintains near-baseline PPL (75.0) while achieving 93–97% top-1 at multi-chunk configs. The optimal λ depends on the application's tolerance for base quality vs parallel coverage.

We also tested **diagonal Jacobian correction** ($J \approx \text{diag}(J)$ instead of $J \approx I$), calibrated offline via Hutchinson probes. The improvement was marginal (< 1% top-1) because the IDN training already makes $J \approx I$, rendering the diagonal approximation redundant. This confirms the principle: **invest in training (making $J \approx I$) rather than inference-time Jacobian estimation.**

4.4 What J ≈ I Really Means: Not Droppable

A natural question: if the last 7 layers have $J \approx I$ (input-invariant residuals), can we simply **drop** them? We compare three approaches:

Method	PPL	ΔPPL%	Top-1	What it does
Sequential (32 layers)	74.8	—	1.000	Run all layers in order
Early exit (25 layers)	**113.6**	**+52%**	**0.766**	**Drop last 7 layers**
All-on-same (25+7)	74.7	−0.1%	1.000	Run 7 on same input
Identity Newton K=1	74.8	−0.1%	1.000	Same, with prefix-sum

Table 4: Early exit vs identity Newton on IDN-trained d32. Dropping the last 7 layers degrades PPL by 52% — they compute important features. But running them on the same input (identity Newton) preserves output exactly. $J \approx I$ means the features are input-invariant, not negligible.

The last 7 layers are **not redundant** — they add important features that improve PPL from 113.6 to 74.8. But $J \approx I$ means their outputs depend negligibly on their inputs (beyond the residual pass-through). They can be evaluated on the **same** input simultaneously, producing identical results to sequential execution.

4.5 Quantifying the Parallelism Opportunity

Identity Newton K=1 does the same total work as sequential (32 layer evaluations). The speedup comes only from **executing the 7 parallel layers simultaneously** rather than sequentially. With careful profiling:

Component	Time	Fraction
Sequential prefix (25 layers)	20.4ms	78%
Last 7 layers (sequential)	7.3ms	28%
Last 7 layers (ideal: 1 layer time)	2.8ms	—
Theoretical total	**23.2ms**	**1.13x speedup**

Table 5: Breakdown of the d32 forward pass. With perfect parallel execution of the 7 identity-Newton layers, the theoretical speedup is 1.13x (saving 17% of total time). Realizing this requires a parallel execution backend.

Our current implementation runs the 7 layers sequentially in a Python loop, yielding no wall-clock speedup ($\sim 1.02\times$). Realizing the 1.13x theoretical speedup requires a parallel execution backend — either batching the 7 blocks into one fused kernel, CUDA graph capture, or multi-device execution. This is an engineering challenge, not an algorithmic one: the identity Newton method has already proven that the 7 layers **can** run on the same input with identical output.

4.6 Layer Fusion: Realizing the Speedup

Since the parallel layers all read the same input, we can **fuse** them into one wider layer by concatenating their weight matrices. For 7 parallel blocks each with 16 attention heads and 8192 MLP hidden units, the fused mega-block has 112 heads and 57344 MLP hidden — one large matmul replaces 7 sequential small ones, naturally saturating GPU tensor cores.

Component	Sequential	Fused	Ratio
7 parallel blocks	7.29ms	3.71ms	1.97x faster
1 block (lower bound)	2.77ms	—	—
Full forward (prefix + suffix)	**23.3ms**	**19.8ms**	**1.18x**

Table 6: Layer fusion on d32 idn=0.5 (seq=25, 7 parallel layers). The fused mega-block runs in 3.71ms — 1.97x faster than 7 sequential blocks, achieving 1.18x end-to-end speedup. Cosine similarity = 0.995.

4.7 Chunkwise Decomposition for Deeper Parallelism

Inspired by DeltaNet's chunkwise parallel algorithm [2], we extend the fusion approach to cover **more** than the last 7 layers. Instead of a single parallel suffix, we divide the suffix into multiple fused chunks, each processed as a mega-block, with additive corrections between them:

$$\boldsymbol{h}^{(c)}_{\text{corrected}} = \boldsymbol{h}^{(c)}_{\text{fused}} + \left(\boldsymbol{h}^{(c-1)}_{\text{corrected}} - \boldsymbol{h}_{\text{prefix}}\right)$$

This cascaded correction propagates the prefix output through each chunk without re-running the blocks — each correction is a single tensor addition. The fused chunks can execute in any order (or simultaneously on multi-GPU).

Config	PPL	ΔPPL%	Top-1	Actual	Speedup
Sequential (32 layers)	74.8	—	1.000	18.6ms	1.00x
seq=24 + 4×fused2	77.1	+3.1%	0.922	14.0ms	**1.33x**
seq=20 + 2×fused6	78.9	+5.4%	0.812	12.3ms	**1.51x**
seq=20 + 1×fused12	81.5	+8.9%	0.773	12.0ms	**1.55x**
seq=16 + 2×fused8	151.8	+103%	0.621	11.2ms	**1.66x**

Table 7: Chunkwise fused parallel on d32 idn=0.5. Quality degrades with more aggressive chunking because the current IDN training targets only the last 3/5/7 layers. Training with coverage of layers 16+ would improve quality at the aggressive configs.

The quality-speed tradeoff forms a clear Pareto frontier: **1.33x at +3% PPL, 1.51x at +5%, 1.66x at +103%**. The quality degradation at seq=16 occurs because the IDN training regularization only covers the last 7 layers (3/5/7 in the current loss). Extending the training loss to penalize identity Newton residuals for 12–16 layers would improve quality at the aggressive configs.

The chunkwise approach connects to DeltaNet's insight [2]: for short recurrences ($N = 4$–8 per chunk), the sequential within-chunk cost is small, while the chunk-level parallelism leverages fused matmuls and tensor cores. Unlike the Blelloch parallel scan (which requires materializing $O(Ld^2)$ Jacobian matrices), the chunkwise method materializes only the fused weight matrices (constant overhead, amortized across all inputs).

4.8 Comparison of All Methods Explored

Method	JVP cost	Output fidelity	Best speedup	Status
Vanilla Jacobi	0	Diverges	—	Failed
FD-Newton	$2 \times$ fwd	Exact	0.35x	Too expensive
VJP-Newton	$2.6 \times$ fwd	Exact	0.31x	Too expensive
Diagonal quasi-DEER	~ 0	Approximate	0.54x	Too noisy
Early exit	0	PPL +52%	1.24x	Quality loss
Identity Newton K=1	0	100% top-1	1.02x	No parallelism
Layer fusion	**0**	**cos=0.995**	**1.18x**	**Real speedup**
Chunkwise fused	**0**	**PPL +3–5%**	**1.33–1.55x**	**Best speedup**

Table 8: All methods tested. Chunkwise fused parallel achieves the best wall-clock speedup by combining identity Newton (algorithmic correctness), layer fusion (GPU efficiency), and chunkwise decomposition (DeltaNet-inspired parallelism).

5 Conclusions

We explored layer-parallel inference for transformers, progressing from DEER/Newton theory to practical chunkwise fusion with training co-design. The key findings:

1. **Exact Jacobian methods don't pay off.** FD-Newton, VJP-Newton, and diagonal quasi-DEER all converge but the Jacobian computation cost (2–2.6 $\times$ forward) exceeds any parallelism savings.
2. **The identity approximation is enough.** Replacing the Jacobian with I reduces Newton's forward substitution to a prefix sum (zero cost). Parallel layers can be evaluated on the **same input**, producing near-identical output.
3. **Training co-design is the multiplier.** Identity-Newton-aware training ($\lambda_{\text{idn}} = 0.5$, 7% overhead) expands the convergence envelope from 1–3 to 7 parallel layers with perfect output fidelity.
4. **Layer fusion converts parallelism to speedup.** Concatenating parallel blocks' weights into one mega-matmul gives **1.18x real wall-clock speedup** (7 blocks in 3.71ms vs 7.29ms sequential), naturally saturating GPU tensor cores.
5. **Chunkwise decomposition enables deeper parallelism.** Splitting the parallel suffix into fused chunks with additive correction extends the approach beyond the last 7 layers, achieving **1.33x at +3% PPL** and **1.55x at +5.4% PPL** on d32.
6. **The layers are not droppable.** Early exit at layer 25 degrades PPL by 52%. The parallel layers compute important but input-invariant features.

The recommended recipe: (1) train with `--identity-newton-reg=0.5` targeting the last N layers; (2) at inference, fuse those N layers into a mega-block and run it on the sequential prefix's output; (3) for more aggressive speedup, chunk the suffix into multiple fused mega-blocks with additive correction.

6 Future Directions

With identity Newton + training co-design validated on 32-layer models, several directions could amplify the speedup:

6.1 Deeper Models

The speedup formula $\frac{L}{S+K\cdot N}$ improves with depth. For $L = 128$ (e.g., LLaMA-3 405B) with 20 parallel layers at $K = 1$: $\frac{128}{108+20} = 1.0x$ — break-even. But the IDN-trained model might tolerate $K = 1$ with 30+ parallel layers, giving $\frac{128}{98} = 1.31x$. Training a d64 or d128 model with identity-Newton reg would test this.

6.2 Larger Parallel Fractions

On d32, $K = 1$ converges perfectly for 7 of 32 layers (22%). Stronger regularization ($\lambda_{\text{idn}} = 2$ –5) or longer training might push this to 10–15 layers (30–47%), where the speedup reaches $\frac{32}{17+15} = 1.0x$ to $\frac{32}{22+10} = 1.0x$ at $K = 1$. The key question: is there a quality floor below which the regularization degrades the model?

6.3 Architectural Co-Design

Parallel attention + MLP blocks (GPT-J/PaLM style, $h_i = h_{i-1} + \text{attn}(h_{i-1}) + \text{mlp}(h_{i-1})$) have inherently smaller layer Jacobians than the standard sequential block. Training such architectures with IDN regularization could enable even more aggressive parallelization.

6.4 Parallel Execution Backend

The most immediate next step: realizing the 1.13x theoretical speedup on actual hardware. Options include (a) batching the 7 parallel blocks into a single large matmul (treating blocks as a batch dimension), (b) CUDA graph capture to eliminate Python loop overhead, (c) a Triton kernel that fuses 7 block evaluations, or (d) multi-device execution where each device runs a subset of blocks. The identity Newton method has already proven the **algorithmic** correctness; what remains is the **systems** engineering.

6.5 Sub-Layer Pipelining

Our profiling shows QKV projections (13% of block time) can overlap with the previous layer's MLP (41% of block time). This orthogonal optimization could yield $\sim 1.3\times$ speedup with zero quality loss, multiplicative with identity Newton's 1.13x for a combined $\sim 1.5\times$.

6.6 Token-Level Combination

Layer-parallel (this work) and token-parallel (Lookahead Decoding [3]) address orthogonal bottlenecks. Combining them could yield multiplicative speedups: $1.13\times$ (layer) $\times$ 1.5–2 $\times$ (token) = 1.7–2.3 $\times$ total.

References

Bibliography

[1] Y. H. Lim, Q. Zhu, J. Selfridge, and M. F. Kasim, "Parallelizing non-linear sequential models over the sequence length," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] Y. Song, C. Meng, R. Liao, and S. Ermon, "Accelerating Feedforward Computation via Parallel Nonlinear Equation Solving," in *International Conference on Machine Learning (ICML)*, 2021.

[3] A. Santilli *et al.*, "Accelerating Transformer Inference for Translation via Parallel Decoding," in *Association for Computational Linguistics (ACL)*, 2023.